DroneRS Embedded / Firmware Team

Three-person beginner onboarding packet

Version 1.0 — August 25, 2026

Your mission: connect a working person-following neural network to safe drone behavior. In this project, "frontend" means embedded C, camera data, messages, control decisions, and hardware integration. It does *not* mean a web user interface.

Your first success should happen before you change code: clone the repository, verify its files, and run the shipped GAP8 application in simulation. Your first programming project then builds a small command sandbox on a laptop. No propellers, motors, or flashing are involved.

1. What Already Works

The active model is `plain_follow`. It accepts one grayscale image and produces information about whether a person is visible, where the person is horizontally, and approximately how large the person appears.

Repository	`https://github.com/DavidLiu2/pytorch-ssd`
Baseline branch	`unstable`
Baseline commit	`b11e9da`
Generated app	`application/`
Input	$1 \times 128 \times 128$ grayscale, 16,384 bytes
Output	14 signed 32-bit integers, 56 bytes
Validated state	GVSOC pass; all 9 layers and final tensor match exactly
Not complete	Live camera path, AI-deck/Crazyflie messaging, control law, and physical flight

Honest boundary: the checked-in neural-network application works in simulation. The repository is not yet a complete person-following drone firmware. Do not describe it as flight-ready.

2. The System in One Picture

Camera frame ⟶ 128x128 grayscale ⟶ GAP8 network ⟶ 14 integers
⟶ decoder ⟶ target message ⟶ safe controller ⟶ Crazyflie

The backend team owns the model and the meaning/scaling of its outputs. Your team owns moving bytes through the system, decoding the agreed contract, making safe decisions, and integrating with hardware.

Vocabulary

Crazyflie	The small research quadcopter and its main flight-control processor.
AI-deck	Expansion board containing the camera, GAP8 processor, and Wi-Fi processor.
GAP8	Low-power microprocessor that runs the neural network.
Firmware	Software compiled for an embedded device rather than a laptop.
GVSOC	A simulator for GAP processors. It lets us test without hardware.
Tensor	A numbered array. Here, the final tensor is a list of 14 integers.
Checkpoint	Saved learned model parameters.

3. Team Responsibilities

Assign one primary owner to each area, but review one another's work.

Person F1	Inference and decoder	Understand the generated API, parse the 14 outputs, and write decoder tests.
Person F2	Control and safety	Turn decoded targets into mock commands; own STOP/lost-target behavior and limits.
Person F3	Integration and tools	Own the command-line demo, logging, setup notes, GVSOC runs, and later hardware bring-up.

Rotate code reviews. Nobody merges their own pull request without another teammate reading and running it.

4. Install This Tech Stack

Use one standard environment so the team sees the same behavior.

Required during week 1

1. **Windows 10/11 with WSL 2 and Ubuntu.** Open PowerShell as Administrator:

   ```
   wsl --install
   wsl --update
   wsl -l -v
   ```

   The Ubuntu entry should show WSL version 2. Official guide: `https://learn.microsoft.com/en-us/windows/wsl/install`

2. **Visual Studio Code for Windows** plus these extensions:

   - WSL (Microsoft)
   - C/C++ (Microsoft)
   - Python (Microsoft)
   - GitHub Pull Requests (optional but helpful)

   Official WSL guide: `https://code.visualstudio.com/docs/remote/wsl`

3. **Docker Desktop** with the WSL 2 engine and Ubuntu integration enabled. Start Docker Desktop before running GVSOC. Official guide: `https://docs.docker.com/desktop/features/wsl/`

4. In the Ubuntu terminal:

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip build-essential
git --version
python3 --version
docker --version
```

Required later, when the physical drone is assigned

Install the Crazyflie client on Windows, not during an unsupervised first session. The hardware owner will also need the Crazyradio USB device and the Bitcraze firmware repositories. Use only the official guides:

- Official Crazyflie getting-started guide
- Official AI-deck getting-started guide
- Official building and flashing guide
- Official Crazyflie app-layer guide

Do not flash custom firmware or power motors until a mentor has identified the exact Crazyflie model, provided a recovery procedure, and approved the test. Remove propellers for every bench test that could arm motors.

5. Clone and Validate the Handoff

Use a path with no spaces. The explicit destination name `pytorch_ssd` matters because some historical scripts expect it.

```
mkdir -p /mnt/c/DroneRS
cd /mnt/c/DroneRS
git clone https://github.com/DavidLiu2/pytorch-ssd.git pytorch_ssd
cd pytorch_ssd
git switch unstable
git pull --ff-only
git rev-parse --short HEAD
```

The commit should be `b11e9da` or a newer handoff commit. If it is older, stop and contact David.

Run the file-integrity check:

```
python3 tools/verify_plain_follow_handoff.py
```

Expected result:

```
plain_follow handoff integrity check: PASS
```

Then start Docker Desktop and run the application in GVSOC:

```
bash ./run_plain_follow_app_val.sh
```

Near the end, look for `PASS: 'final' matches exactly`. Save the terminal output in your first issue or pull request.

6. The Generated Network Interface

Read these files in order:

1. `application/README.md`
2. `application/validation/manifest.json`
3. `aideck_val_main_plain_follow.c`
4. `application/inc/network.h`

The current validated harness performs this sequence:

```
mem_init();
network_initialize();

uint8_t *buffer = pi_l2_malloc(412000);
/* Copy exactly 128 * 128 grayscale bytes into buffer. */
network_run(buffer, 412000, buffer, 0, 1);

/* Read 14 int32_t values from the output buffer. */
network_terminate();
```

The output head is `xbin9_size_bucket4`:

Indices 0–8	9 x-position logits	Choose the largest value; its bin center estimates horizontal position from left (-1) to right ($+1$).
Index 9	Visibility logit	Backend and firmware teams must agree on the deployed threshold/scaling.
Indices 10–13	4 size logits	Choose the largest value; centers are 0.125, 0.375, 0.625, and 0.875.

Do not edit files under `application/` by hand. They are generated artifacts. Build integrations around the public functions in `network.h`.

7. Initial Project: Follow Command Sandbox

Goal: build and test the logic between the neural-network output and a future flight controller, entirely on a laptop. The sandbox reads 14 integers, decodes a target, and prints a mock command. It must never connect to hardware.

Create this folder on a team branch:

```
student_projects/frontend_follow_command/
  decoder.py
  controller.py
  main.py
  test_decoder.py
  test_controller.py
  README.md
```

Decoder contract

Input: a list of exactly 14 integers. Output: a small record containing:

- `visible`: boolean
- `x_bin`: integer from 0 through 8
- `x_value`: bin center from approximately -0.889 through $+0.889$
- `size_bucket`: integer from 0 through 3
- `size_value`: one of 0.125, 0.375, 0.625, 0.875

For this first sandbox only, use `output[9] > 0` as the mock visibility rule. Label it clearly as a temporary rule that must be confirmed with the backend team before embedded integration.

Start with the real validated vector:

```
4632 13262 4633 -2422 -3479 -5390 2962
-1854 -11170 5303 -7980 3540 4466 -43
```

It should decode to visible, x-bin 1 (left, about -0.667), and size bucket 2 (0.625).

Mock controller contract

Return exactly one command. Evaluate rules from top to bottom:

Not visible	`STOP_AND_SEARCH`
$x < -0.25$	`TURN_LEFT`
$x > +0.25$	`TURN_RIGHT`
Centered and size < 0.375	`MOVE_FORWARD`
Centered and size > 0.625	`MOVE_BACKWARD`
Otherwise	`HOLD`

These are symbolic commands, not motor values. The validated vector should produce `TURN_LEFT`.

Divide the work

- F1 implements `decoder.py` and its tests.
- F2 implements `controller.py` and its tests.
- F3 implements `main.py`, input validation, logging, and the README.
- All three review and run the combined program.

Definition of done

- At least 6 tests: validated vector, invisible, far-left, far-right, too-small, too-large, and malformed input.
- Malformed inputs fail with a useful message, not a mysterious stack trace.
- The README explains the 14 fields and states that commands are simulations only.
- The team opens one pull request containing terminal test output and a three-minute demo.
- No files under `application/` are changed.

8. Suggested Two-Week Schedule

Day 1	Install tools, clone, integrity check, GVSOC run.
Day 2	Read the four interface files and draw the data flow together.
Days 3–4	Implement decoder, controller, CLI, and tests in parallel.
Day 5	Integrate, review, demo, and merge the sandbox.
Week 2	Port the decoder and state machine to host-compiled C with unit tests. Do not flash yet.
End of week 2	Joint interface review with backend team: output layout, scaling, threshold, message fields, units, and update rate.

9. Git and Collaboration Rules

Create one branch per task:

```
git switch unstable
git pull --ff-only
git switch -c frontend/<your-name>-decoder
```

Before asking for review:

```
git status
python3 -m unittest discover -s student_projects/frontend_follow_command -v
git diff --check
```

Keep pull requests small. Explain what changed, how you tested it, and what is still uncertain. Never commit batteries of generated build files, private photos, passwords, radio addresses, or virtual environments.

10. Copy-Paste AI Starter Prompt

> You are tutoring a team of first-year college students who are new to programming and embedded systems. We are working in the `pytorch-ssd` repository on branch `unstable`, baseline commit `b11e9da`. Our first project is a laptop-only Follow Command Sandbox. It must not connect to or flash drone hardware.
>
> The real model output is 14 signed integers: indices 0–8 are x-position logits, index 9 is a visibility logit, and indices 10–13 are size-bucket logits. For this sandbox only, visible means value 9 is greater than zero. Decode x and size with argmax and the centers documented in our onboarding packet. Then apply symbolic STOP/LEFT/RIGHT/FORWARD/BACKWARD/HOLD rules.
>
> Please act as a patient teacher. Explain every new term briefly, give one small step at a time, and tell us how to test each step. Reuse the repository's documented contract; do not invent a different API. Do not edit generated files under `application/`. Do not suggest flashing, arming, or running motors. If you need repository context, ask us to paste the relevant file instead of guessing.
>
> First response only: (1) restate the data flow in plain English, (2) propose the five-file project structure, (3) list at least six tests, and (4) give us only the first implementation step for `decoder.py`. Stop after that so we can try it and report the result.

11. How to Ask for Help

Send all of the following:

- What you expected.
- The exact command you ran.
- The complete error text as copyable text.
- Output of `git status -short` and `git rev-parse -short HEAD`.
- One thing you already tried.

Stop after 30 minutes on the same error. Ask a teammate, then the project lead. Do not randomly reinstall tools or delete folders to make an error disappear.

12. First-Day Checklist

- [ ] I can open the repository through VS Code's WSL mode.
- [ ] `git rev-parse -short HEAD` shows the expected baseline or newer.
- [ ] The integrity checker prints **PASS**.
- [ ] The GVSOC command prints an exact final-tensor pass.
- [ ] I can explain the camera-to-command data flow without notes.
- [ ] I chose F1, F2, or F3 and opened my first GitHub issue.
- [ ] I understand that no physical flight is part of the initial project.

Source of truth: repository README, `application/validation/manifest.json`, and reviewed code. AI output is assistance, not evidence.